CONVERSIONE BINARIO – ESADECIMALE

0110101101010 $\xrightarrow{\text{suddivido in nibbles}}$ $\underbrace{0}_{0}\ \underbrace{1101}_{D}\ \underbrace{0110}_{6}\ \underbrace{1010}_{A}$ $\longrightarrow$ 0D6A

SOMME

1 + 0 = 1 no riporto   1 + 1 = 0 riporto 1   1 + 1 + 1 = 1 riporto 1

PRODOTTO

$$
\begin{array}{r}
1011101 \times 1011 = \\
\hline
{}^{1}1{}^{1}0\,1101 \\
{}^{1}1011101- \\
{}^{1}0000000-- \\
1011101--- \\
\hline
11110111 1
\end{array}
$$

MULTIPLICAZIONE PER NUMERI BINARI IN FORMA 2^n

$10001 \cdot 100 = 10001\underline{00}$ $11\boxed{001} / 100 = 110$

2^2 → aggiungo 2 "0"   → con resto 01

OVERFLOW: quando il risultato di un'operazione a n bit è di dimensione maggiore di 2^n

NOTAZIONE SEGNO-VALORE ASSOLUTO

101101 → 1 indica segno "–" ⇒ –13

→ 0 indica segno "+" ⇒ +13

NOTAZIONE COMPLEMENTO A DUE

$101101 = \underbrace{100000}_{32} - \underbrace{001101}_{13} = 19$

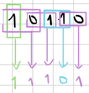
BCD (BINARY-CODED-DECIMALS)

$\underbrace{00000001}_{1}\ \underbrace{00000111}_{7} = 17$

BINARIO – CODICE GRAY

Il codice Gray è utile perché le codifiche dei numeri sono configurazioni adiacenti. Si considerano partendo da sinistra, le coppie di valori: se sono uguali metto 0, se sono diversi metto 1. Nota: il primo bit da sinistra, detto bit significativo, rimane uguale

Es.



DEC. BIN. GRAY

In questo modo ottengo che

0	000	000
1	001	001
2	010	011
3	011	010
	...	

→ adiacenti

LEGGE DI DE MORGAN: $!(x+y) = !x \cdot !y$ $!(x+y) = !x + !y$

T. DI SHANNON: $A = X \cdot A + !X \cdot A$ $A = (X+A) \cdot (!X+A)$ ← A valore fisso (0 o 1)

MINTERMINE: prodotto della sintesi SP

MAXTERMINE: somma della sintesi PS

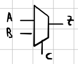
MUX 2

Rete con 2 ingressi A e B, un segnale di controllo C e una uscita Z.

Se C = 0 ⟶ passa A ⟶ Z = A

Se C = 1 ⟶ passa B ⟶ Z = B

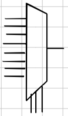
$Z = A!C + BC$



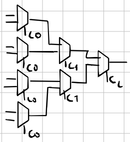
MUX 2^n

Rete con 2^n ingressi, n segnali di controllo, un uscita. Ogni configurazione di segnali di controllo discrimina il passaggio di uno specifico ingresso.

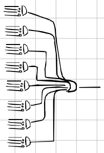
MUX 8



Si può costruire un MUX a 2^n vie come albero di MUX, in cui ogni livello è pilotato da un bit di controllo, o come una linea di AND tra ogni ingresso e tutti i bit di controllo



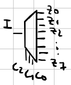
o



↑ ingressi messi a caso, hanno una logica

DEMUX A 2^N VIE

Duale del MUX, ha 1 ingresso, n segnali di controllo e 2^n uscite. a seconda della configurazione dei bit di controllo, discrimina l'uscita da cui far uscire l'ingresso

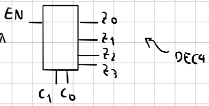


← DEMUX 8

DECODER

Può essere visto come un DEMUX con segnale in ingresso 1 (o opzionalmente ENABLE). Decide l'uscita da mandare a 1 (ovvero l'uscita da attivare) a seconda della configurazione dei bit di controllo

serve solo ad abilitare/disabilitare il DECODER



ENCODER

Anche qui un solo segnale d'ingresso =1, 2^n bit di controllo, n uscite. A seconda del bit di controllo =1, viene prodotta una certa configurazione di uscita.

t.d.v.

classico esempio di funzione incompleta

c_3	c_2	c_1	c_0	z_1	z_0
0	0	0	0	-	-
0	0	0	1	0	0
0	0	1	0	0	1
0	0	1	1	-	-
0	1	0	0	1	0
...				-	-
1	0	0	0	1	1
...				-	-

Es. ENC4

HALF ADDER

Rete con 2 ingressi (2 bit) e 2 uscite (S (somma) e R (riporto)).

T.D.V.

A	B	S	R
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

→ $S = \overline{A!B + !AB}$ → equivalente a $A \oplus B$

└ A XOR B

→ $R = AB$

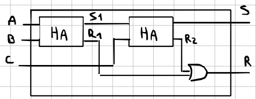
FULL ADDER:

Come un half adder ma tiene conto di un bit di ingresso in più (il bit di carry-in, il riporto dalla operazione precedente)

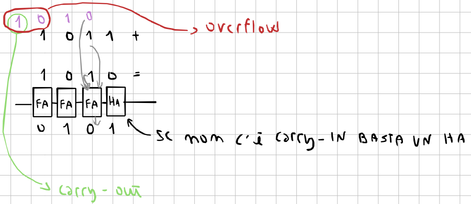
T.D.V.

A	B	C	S	R
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

un FA può essere realizzato con due HA



⇒ un ADDER può essere realizzato con un HA e n-1 FA



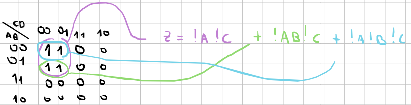
RETE BINARIO-GRAY

Si può dimostrare tramite sintesi della tabella binario-gray che (a 3 bit) per il bit significativo $y_n = b_n$, per gli altri $y_n = b_n \oplus b_{n+1}$

BIN.	GRAY
000	000
001	001
010	011
011	010
...	
$b_2 b_1 b_0$	$y_2 y_1 y_0$

SINTESI MINIMA: Processo per ottenere espressioni non ridondanti (e dunque una rete di costo minimo).

Dato una espressione F, es. $F = abc + cd + da$, tutti i termini prodotto si dicono implicanti



AB \ CD	00	01	11	10
00	1	1	0	0
01	1	1	0	0
11	0	0	0	0
10	0	0	0	0

z = !A!C + !AB!C + !A!B!C

si direbbe inoltre essenziale se raccogliesse
un 1 che non può essere raccolto altrimenti

$\downarrow$
sono tutti implicanti

- implicante primo
- implicante ma non essenziale e non primo
- implicante ma non essenziale e non primo

nel caso della sintesi PS si parla di implicati (le somme) e non di implicanti.

ALTRE ALGEBRE

NOR : OR negato ⇒ vale 1 solo quando gli ingressi sono tutti 0

NAND : AND negato ⇒ vale 0 solo quando gli ingressi sono tutti 1

Algebra di Boole {vero, falso} {e, o, non}

Algebra di Shannon $\{0,1\}\{+, \cdot, '\}$ (la traduzione di quella di Bool)

Algebra del NAND $\{0,1\}\{\uparrow\}$ ⟶ algebra funzionalmente completa

Algebra del NOR $\{0,1\}\{\downarrow\}$ ⟶ algebra funzionalmente completa

Algebra lineare $\{0,1\}\{\oplus, \cdot\}$ ⟶ algebra funzionalmente completa

Algebra dello XOR $\{0,1\}\{\oplus\}$ ⟶ algebra non funzionalmente completa, serve anche l'AND (algebra lineare)

Un algebra si dice funzionalmente completa se è possibile effettuare tutte le operazioni dell'algebra booleana tramite i propri operatori)

SHANNON	NAND	NOR
$!A$	$A \uparrow A$	$A \downarrow A$
AB	$(A \uparrow B) \uparrow (A \uparrow B)$	$(A \downarrow A) \downarrow (B \downarrow B)$
$A+B$	$(A \uparrow A) \uparrow (B \uparrow B)$	$(A \downarrow A) \downarrow (B \downarrow B)$

NAND e NOR richiedono un minor numero di transistor per essere realizzati

SINTESI SP ⟶ SINTESI A NAND

$F = a \cdot b + c' \cdot d + c \cdot f' + y$

$\downarrow$ definizione di ↑ (è un AND (·) negato)

$F = (a \uparrow b)' + (c' \uparrow d)' + (c \uparrow f')' + y$

↳ NAND = NOT AND ⇒ $!(!A\,!B)$

$\downarrow$ II° l. di De Morgan $!(a+b) = !a\,!b$

$F = ((a \uparrow b) \cdot (c' \uparrow d) \cdot (c \uparrow f') \cdot y')'$

$\downarrow$ definizione di ↑ (è un AND (·) negato)

$F = (a \uparrow b) \uparrow (c \uparrow d) \uparrow (c \uparrow f') \uparrow y'$

XOR ⟶ NAND

Definizione di XOR: $a \oplus b = a!b + !ab$

$F = ab' + a'b \equiv F = a \oplus b$

$\downarrow$ "0" ⇒ non cambia l'espressione

$F = ab' + a'b + a'a + b'b$

$\downarrow$ raccolgo

$F = a(b'+a') + b(a'+b')$

$\downarrow$ $\underline{b'+a' = (b \cdot a)'}$

$F = a(b \cdot a)' + b(ab)'$

$\downarrow$ $\underline{\text{def. di } \uparrow}$

$F = (a \uparrow (b \uparrow a)) \uparrow (b \uparrow (a \uparrow b))$

Analogamente:

SINTESI PS ⟶ SINTESI A NOR

$F = (a'+b'+c') \cdot (d'+c') \cdot f' \cdot y$

$\downarrow$

$F = (a' \downarrow b' \downarrow c)' \cdot (d' \downarrow c)' \cdot f' \cdot y$

$\downarrow$

$F = ((a' \downarrow b' \downarrow c) + (d' \downarrow c) + f + y')'$

$\downarrow$

$F = (a' \downarrow b' \downarrow c') \downarrow (d' \downarrow e) \downarrow f \downarrow y'$

XNOR: $A \odot B = (A + !B) \cdot (!A + B)$

SINTESI A XNOR ⟶ SINTESI A NOR

$F = (a+b') \cdot (a'+b)$

$\downarrow$

$F = (a+b') \cdot (a'+b) \cdot (a'+a) \cdot (b'+b)$

$\downarrow$

$F = (a+a'b') \cdot (b+a'b')$

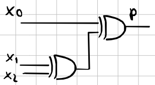
$\downarrow$

$F = (a+(a' \cdot b')) + (b+(a' \cdot b'))$

$\downarrow$

$F = (a \downarrow (a \downarrow b)) \downarrow (b \downarrow (a \downarrow b))$

UTILIZZO DELLO XOR - GENERATORE DI PARITÀ

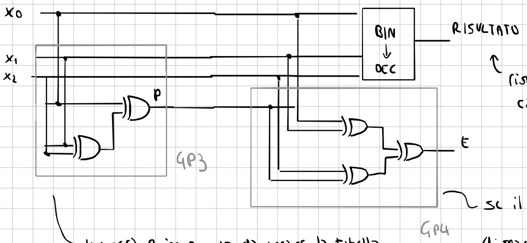


p x2 \ x1 x0	00	01	11	10
00	0	E	3	E
01	E	5	E	6
11	4	E	7	E
10	E	1	E	2

Supponiamo di mandare 3 bit $x_2 x_1 x_0$, che restituiscano (non rilevante) il corrispondente numero decimale. Se x_2 commuta per via di un disturbo, si corrompe l'uscita (es. 01$\boxed{1}$ → 3, 01$\boxed{0}$ → 2).

Per ovviare al problema, introduciamo un bit p e supponiamo che il mittente mandi solo configurazioni a distanza 2 o maggiore. Se il bit x_2 arriva commutato, allora si finisce in una cella di errore. Notiamo che le celle di errore sono in corrispondenza di configurazioni di ingresso in cui il numero di 1 è dispari. Possiamo generare il p necessario per la creazione di questa tabella a partire da degli XOR, come mostrato sopra. Tale generatore restituisce 1 se la configurazione di ingresso ha un numero dispari di 1 (ricorda che XOR è 1 solo se il numero di 1 è dispari).

Mandando poi x_1, x_2, x_0 e p in nel circuito ricevente e in un generatore di parità a 4 bit, otterremo se la configurazione iniziale è giusta



risultato, valuta E per capire se è sbagliato

se il numero di 1 è dispari, allora dà 1 (= errore)

genera p in modo da creare la tabella

p x2 \ x1 x0	00	01	11	10
00	0	E	3	E
01	E	5	E	6
11	4	E	7	E
10	E	1	E	2

SINTESI A DECODER

Ricordando che il decoder attiva una sola uscita alla volta in base alla configurazione dei segnali di controllo, possiamo realizzare la sintesi di qualsiasi rete tramite decoder

es. full'adder

Espressioni del full-adder: $S = !A!BC + !AB!C + A!B!C + ABC$

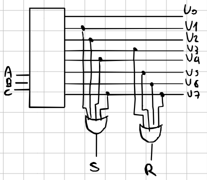
$R = !ABC + A!BC + AB!C + ABC$

ABC ingressi del full'adder

$$S = \overbrace{!A!BC}^{m_1} + \overbrace{!AB!C}^{m_2} + \overbrace{A!B!C}^{m_4} + \overbrace{ABC}^{m_7}$$

$$R = \underbrace{!ABC}_{m_3} + \underbrace{A!BC}_{m_5} + \underbrace{AB!C}_{m_6} + \underbrace{ABC}_{m_7}$$

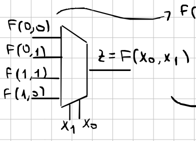
Ad ogni configurazione di ingresso (eccetto !A!B!C, non rilevante) corrisponde un mintermine => il DEC può essere visto come generatore di mintermini: a una specifica configurazione d'ingresso attiva una specifica configurazione di uscita, che corrisponde a un mintermine, dopodiché si fa l'OR tra mintermini (o l'AND tra maxtermini nel caso di espressione ps)



MUX PROGRAMMABILE

Di base un MUX decide, tramite la configurazione di segnali di controllo, quale dei 2^n ingressi far passare.

In una TDV, a seconda della configurazione degli ingressi viene prodotta una certa uscita. Se non conosciamo a priori l'uscita, potremmo semplicemente dire che a seconda della configurazione degli ingressi viene scelta una certa funzione. Ad esempio, per la configurazione 000 viene scelto l'ingresso 1, che può essere una funzione. Dunque stiamo scegliendo che funzione mandare in uscita ⇒ è il comportamento di un MUX



→ F(0,0) nel senso che viene selezionata se x_1 e x_0 sono 00

x_1 x_0	z
0 0	F(0,0)
0 1	F(0,1)
1 0	F(1,0)
1 1	F(1,1)

↑ bit di programmazione

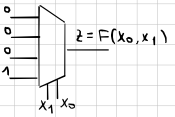
È programmabile nel senso che, cambiando gli F in entrata, cambia la funzione (la TDV) realizzata dal MUX. Vediamo due esempi. Ho un MUX4. Voglio realizzare un AND. Dunque la TDV è

x_1 x_0	z
0 0	F(0,0)
0 1	F(0,1)
1 0	F(1,0)
1 1	F(1,1)

⇒

x_1 x_0	z
0 0	0
0 1	0
1 0	0
1 1	1

⇒



non significa che reti più complesse debbano essere create a partire da queste

Se ora voglio realizzare un OR

anche a più bit ovviamente

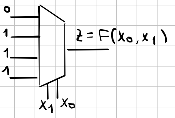
(sono solo due esempi, si può realizzare qualsiasi rete)

x_1 x_0	z
0 0	F(0,0)
0 1	F(0,1)
1 0	F(1,0)
1 1	F(1,1)

⇒

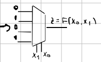
x_1 x_0	z
0 0	0
0 1	1
1 0	1
1 1	1

⇒



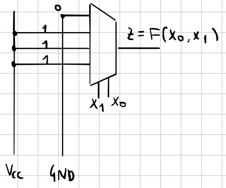
Es:

x_1 x_0	
0 0	0
0 1	1
1 0	0
1 1	1



Ho sintetizzato la rete tramite MUX ⇒ con un solo MUX posso creare tante funzionalità

Fisicamente, zeri e uni in ingresso sono dati da VCC e GND



Quindi si può realizzare qualsiasi rete mettendo come ingressi del MUX le uscite desiderate, e come bit di controllo (che scelgono le uscite) gli ingressi delle funzioni.

SINTESI TRAMITE ALBERO DI MUX

Si basa sul T. di espansione di Shannon,

$f(x_1, x_2, \dots) = \bar{x}_1 \cdot f(0, x_2, \dots) + x_1 \cdot f(1, x_2, \dots)$

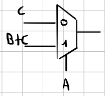
Ad esempio, si vuole realizzare la funzione

$Y = A \cdot B + C$ scelgo ad esempio A come ingresso significativo

per Shannon $Y = !A\,(0 \cdot B + C) + A\,(1 \cdot B + C) = !A\,(C) + A \cdot (B + C)$

Se A è =0 (!A) dobbiamo far passare C, se A=1 (A) dobbiamo far passare B+C

⟹



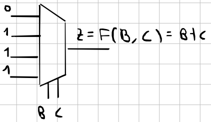
B+C è una rete, e come visto possiamo sintetizzarla tramite MUX

(B+C) è l'OR, non un Adder ovviamente

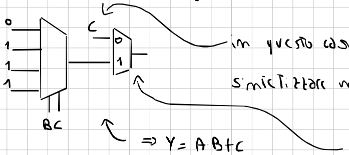
BC	z
0 0	0
0 1	1
1 0	1
1 1	1

B+C

⟹



⟹



in questo caso è solo c dunque non è necessario sintetizzare una funzione tramite Mux

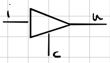
⟹ $Y = A \cdot B + C$

"MUX padre"

senza stati

Quelle viste finora sono reti combinatorie. Vediamone alcune usate a livello pratico.

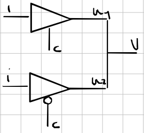
BUFFER TRISTATE



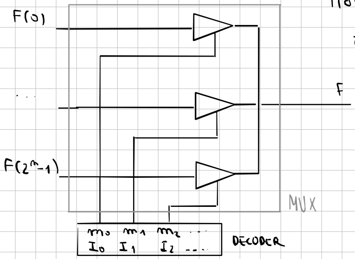
c \ i	0	1
0	Hi-Z	Hi-Z
1	0	1

Componente che lavora in logica ternaria e fa passare il valore d'ingresso se c (segnale di controllo) =1, altrimenti scollega l'uscita (non fa passare il segnale (returna Hiz, "alta impedenza")

Possiamo realizzare un MUX2 tramite buffer



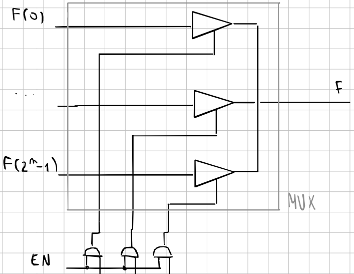
→ problema: se più di un uscita è collegata (≠ Hiz) allora si "mischiano" le due uscite (non consentito in logica binaria) ("cortocircuito")



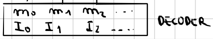
Tramite DEC attivo una sola uscita corrispondente al mintermine desiderato (es. 001 → m_1 (mintermine dello uscita F(1)) ⇒ viene attivato solo il buffer corrispondente a tale uscita, che farà passare il valore di F(1) e solo il valore di F_1 ⇒ no cortocircuito

Altro problema: se dopo voglio far passare F(2), rischio che l'uscita dal buffer di F_1 sia ancora chiusa. Il tempo per aprire un'uscita (0/1 → Hiz) è maggiore del tempo necessario per chiudere un uscita (Hiz → 1/0) ⇒ per un certo istante, F(1) e F(2) saranno in cortocircuito.

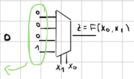
Risolvo disabilitando tutte le uscite per un istante tramite ENABLE = 0, poi si chiuderà solo quella corrispondente al mintermine attivo (es. F_2)



I MUX realizzati tramite buffer tristate sono detti distribuiti, vengono usati ad esempio nel bus dati della CPU, son utili quando un alto numero di componenti deve accedere a un unico canale di comunicazione



Es. per realizzare un AND

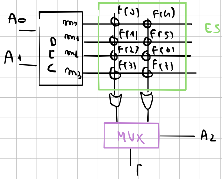


immagazzino questi

MEMORIE

I bit di programmazione sono sempre gli stessi per specifiche funzioni

⇒ a tale scopo possono essere memorizzati da dispositivi di memoria, dette ROM (read-only-memory). È poi possibile andare a selezionare specifici bit di programmazione (una specifica funzione) tramite DECODER e bit di indirizzo

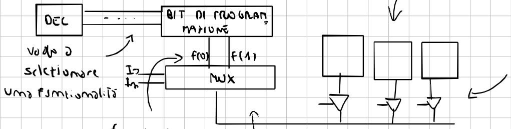


ES. F = ADDER

possono avere tante funzioni memorizzate

ES. DI ARCHITETTURA (SEMPLIFICATA)

componenti



vado a selezionare una funzionalità

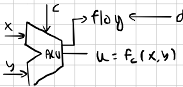
i componenti accedono o meno (tramite buffer tristate) al dato elaborato

funzioni come ingressi del MUX programmabile

MUX calcola la funzione selezionata tramite ingressi come bit di controllo (I_0 e I_1)

ALU (Arithmetic Logic Unit) Δ

Riceve come ingresso due ingressi x e y e un bit di controllo che discrimina che operazione (aritmetica o logica) eseguire.



flag ← da informazioni sul risultato (es. se si seleziona come operazione l'addizione, flag potrebbe dare 1 in caso di riporto)

ES:

CI = Carry In (esempio)

$C_2 C_1 C_0$ \ CI	0	1
0 0 0	$x+y$	$x+y+1$
0 0 1	$x-y-1$	$x-y$
...	...	...

op

operazioni aritmetiche

C_3 ← altro generico segnale di controllo

$C_2 C_1 C_0$ \ C_3	0	1
0 0 0	$x \oplus y$	x OR y
0 0 1	...	...

operazioni logiche

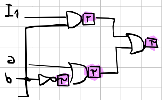
PROBLEMI NELLE RETI COMBINATORIE

PROBLEMA 1 - NON SIMULTANEITÀ DEGLI INGRESSI

Gli ingressi non possono cambiare nello stesso istante. Es: (01 → 10) può avvenire tramite 01 → 11 → 10 o 01 → 00 → 10 (cambia prima un bit dell'altro). Durante tale istante, si attraversa una configurazione indesiderata, che può portare a una uscita indesiderata. Si definisce corsa l'insieme delle configurazioni assunte passando da una configurazione a un'altra (es: 01 → 11 → 10). Si definisce corsa critica se durante la corsa l'uscita assume valori indesiderati (glitch)

PROBLEMA 2 - RITARDI E TRANSITORI

Ogni elaborazione richiede tempo → aggiunge un ritardo τ. La somma dei ritardi accumulati da ogni gate di una rete definisce il "transitorio" della rete. Durante il transitorio, l'uscita potrebbe assumere valori indesiderati



A transitorio estinto, ovvero dopo un tempo pari alla somma dei ritardi, l'uscita rispecchierà effettivamente l'uscita desiderata.

Si dice alea un cambiamento indesiderato dell'uscita durante il transitorio.

Alea statica → l'uscita varia una sola volta durante il transitorio

Alea dinamica → l'uscita varia più volte durante il transitorio

Le reti SP e PS presentano solo (eventualmente) alee statiche.

Le alee statiche si dividono in

- Alee di tipo 1: l'uscita, che dovrebbe essere 1, commuta a 0 durante il transitorio. Si verifica solo in reti SP. Per individuarle, basta trovare due "1" sulla mappa adiacenti ma non coperte da un unico implicante. Si risolve l'alea andando a introdurre tale implicante nella espressione

c\ab	00	01	11	10
0	0	0	1	1
1	0	1	1	0

⇒ $U = ac + bc$ → due uni adiacenti non coperti da un singolo implicante

↓

c\ab	00	01	11	10
0	0	0	1	1
1	0	1	1	0

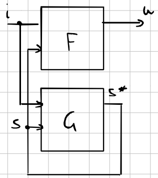
$U = ab + bc + ab$ ⇐ alea statica ⇐

↳ da qui si può realizzare uno schematico senza alee

- Alee di tipo 0: l'uscita, che dovrebbe essere 0, commuta a 1 durante il transitorio. Si verifica solo in reti PS. Per individuarle, basta trovare due "0" sulla mappa adiacenti ma non coperte da un unico implicato. Si risolve l'alea andando a introdurre tale implicato nella espressione

RETI SEQUENZIALI

Rete in cui è presente una retroazione (rete dipendente dal tempo) (nel senso che l'uscita dipende dallo stato, che dipende dal tempo).



Si definisce G la parte della rete che elabora lo stato.

Si definisce F la parte di rete che elabora l'uscita.

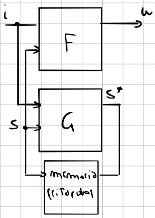
Una rete si dice "di Mealy" se l'uscita dipende sia dagli ingressi che dalle uscite.

Una rete si definisce "di Moore" se l'uscita dipende solo dallo stato.

Rete sequenziale asincrona: rete che ri-elabora lo stato ogni volta che τ_a

si estingue ⇒ non è in grado di riconoscere due configurazioni di ingresso identiche. Es: arriva $i = 1010$ → G lo elabora con tempo T_G ⇒ lo rimanda in ingresso a se stessa appena finisce T_G (anche se magari l'ingresso è sempre lo stesso) → viene riaggiornato immediatamente lo stato

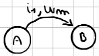
Rete sequenziale sincrona: viene introdotto un ritardo esplicito (dato dalla frequenza di clock, $> T_G$) in modo da poter controllare ogni quanto viene ri-calcolato lo stato. In realtà, come si vedrà più avanti, tale comportamento è attuato da un FFD, che memorizza lo stato futuro (quello appena elaborato) ma rilascia lo stato presente, ad ogni ciclo di clock.



Dunque il transitorio T_G non impatta le reti sequenziali sincrone (se T, il periodo di clock, è $> T_G$) ma impatta le reti sequenziali asincrone

GRAFO: sistema grafico di rappresentazione di un automa

- grafo con rete di Mealy: l'uscita dipende sia dagli ingressi che dagli stati. "Se sono in A e arriva un ingresso i_1, l'uscita varrà u_m e mi porterò nello stato B"



- grafo con rete di Moore: l'uscita dipende solo dallo stato in cui ci si trova ⇒ a ogni stato corrisponde una uscita, tramite gli ingressi si passa di stato. "Se sono in A, l'uscita vale 00. Per passare in B, in cui l'uscita vale 01, deve arrivarmi un ingresso i_1."

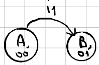


TABELLA DI FLUSSO: tabella che descrive il comportamento dell'automa

TDF di Mealy

	i_1	i_2 ...	i_m
S_1	S_2, U_k	...	...
S_2 ⋮	...	S_5, U_i	...
S_m	...	...	...

← ingressi

TDF di Moore

ingressi

	i_1	i_2 ...	i_m
S_1	S_2	...	...
S_2 ⋮	...	S_5	⋮
S_m	...	...	...

u
U_1
U_2 ⋮
U_m

↑ stati

stato in cui si va (stato futuro) e uscita

↑ stati

stato futuro

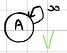
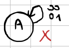
↑ uscite

una per ogni stato

andare in S_5 implica automaticamente che l'uscita vale U_5

SINTESI DI RETI SEQUENZIALI ASINCRONE

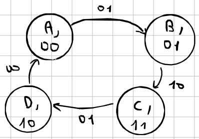
1) Realizzazione d.d.s. (diagramma degli stati

- identifico il minor numero di stati necessari
- realizzazione di un d.d.s. primitivo, ovvero in cui ogni stato è stabile per una sola configurazione di ingresso (es.  )
- si sceglie tra modello di Mealy o di Moore

2) Si ricava la tabella di flusso a partire dal d.d.s

3) si codifica ogni stato in configurazioni binarie. Qui bisogna stare attenti: come visto, si possono avere corse critiche quando due configurazioni sono a distanza > 1. Es. devo andare da 01 a 11 => no problem (deve cambiare solo 1 bit); devo andare da 01 a 10 => problema, i bit non cambiano simultaneamente, si rischia una corsa critica. Visto che il passaggio da uno stato a un altro è a tutti gli effetti una corsa (c'è un passaggio da una configurazione a un'altra) bisogna far sì che stati adiacenti (connessi tra di loro tramite una o più frecce nel d.d.s.) abbiano configurazioni adiacenti, in modo che il passaggio dall'uno all'altro non possa rappresentare una corsa critica.

ES.



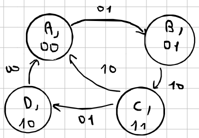
A è adiacente (collegato) a B e D

=> A deve essere codificato adiacente

ES. A = 00 => B = 01
=> D = 10

In questo modo, visto che da A si può andare solo in B o C, non ci saranno corse critiche (sono a distanza 1.

Nota: in generale se uso n bit per la codifica ogni configurazione può essere adiacente a 2 altre configurazioni.



Se ad esempio A è connesso a C, si potrebbero verificare corse critiche nel passaggio C-A, ma allo stesso tempo è impossibile codificare A,B,C,D a 2 bit e far sì che siano tutti e 3 con configurazioni adiacenti. Si possono attuare due metodi di risoluzione:

- codificare a 3 bit => ogni stato può dunque avere 3 configurazioni adiacenti

(es. A=000 → B=001 ← Tutti e 3 adiacenti ad A, ma è poi
↳ C=010 necessario introdurre nuovi stati per i
↳ D=100 passaggi B->C, C->D (che sono a distanza
>1 tra di loro (corse critiche))

ES. E = 011 F = 110 B->C->D => B->E->C->F->D

- semplificare a intuito il d.d.s.

Dunque:

da verificare

d.d.s. primitivo => risolve la non-simultaneità dei bit di ingresso

risoluzione corse critiche => risolve la non-simultaneità dei bit di stato

- Si va dalla tdf alla tdt (nota: tdt = tabella delle transizioni, è la tabella di flusso ma con gli stati codificati (es. in cui A = 000, ...)

4) Si ricava la t.d.v. (o mappe di karnaugh) per ogni variabile di stato
(e per l'uscita)

- dalle tabelle si ricavano poi le espressioni di ogni variabile andando a cerchiare i raggruppamenti (di 1 per SP, di 0 per PS)

uscite

ES.

$y_1y_0 \backslash x_1x_0$	00	01	11	10
00	0	1	1	0
01 11	1	1	0	0
10	0	0	0	0

mappa di Y_0

↳ bit più a destra delle configurazioni di stato futuro (es. $A = \overset{}{00}$ y_1 y_0 ↳ stiamo facendo la tabella della sola Y_0)

$\Rightarrow Y_0 = !x_1 !y_0 y_1 + x_0 !y_0 !y_1$

y_0y_1	z
...	...
...	...
...	...

$\Rightarrow z = \dots$

- rimozione di eventuali alee statiche

$y_1y_0 \backslash x_1x_0$	00	01	11	10
00	0	1	1	0
01 11	1	1	0	0
10	0	0	0	0

$\Rightarrow Y_0 = !x_1 !y_0 y_1 + x_0 !y_0 !y_1 + !x_1 x_0 !y_1$

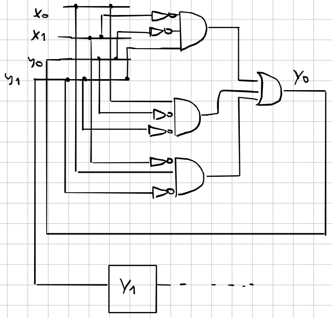
4) Dalle espressioni si disegna lo schematico, riportando opportunamente le retroazioni

ES:

$y_1y_0 \backslash x_1x_0$	00	01	11	10
00	0	1	1	0
01 11	1	1	0	0
10	0	0	0	0

$Y_0 = !x_1 !y_0 y_1 + x_0 !y_0 !y_1 + !x_1 x_0 !y_1$

↳

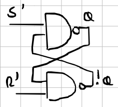


ANALISI DI RETI ASINCRONE ⟶ si procede nel modo opposto schematico ⟶ dds

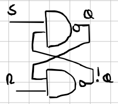
LATCH SR (analisi reperibile nella lezione del 18/11/25)

Rete con due ingressi, S (set) e R (reset). S e R non possono mai essere 11. Se S=1, R=0 ⇒ l'uscita Q=1 (set). Se S=0, R=1 ⇒ l'uscita Q=0 (reset). Se S=0 e R=0 l'uscita Q rimane allo stato precedente.

Il Latch SR può essere sintetizzato sia tramite NAND sia tramite NOR



o

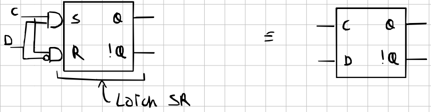


!Q è Q negato, S' e R' sono S e R negati

Il Latch SR è una rete sequenziale asincrona che di base segue il modello di Mealy (Q "dipende" dall'ingresso) ma che a transitori estinti si comporta secondo il modello di Moore (Q "dipende solo" dallo stato)

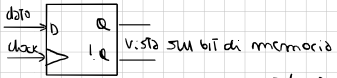
Latch CD (analisi reperibile nella lezione del 18/11/25)

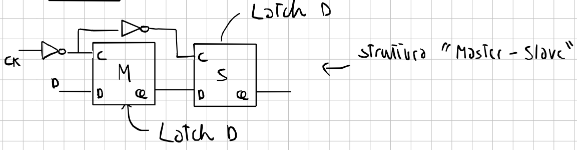
Costruito a partire da un Latch SR, ha due ingressi C e D. Quando C=1, la rete campiona (memorizza) il valore di D. Quando C=0 la rete mantiene in memoria il valore precedente (Q = valore precedente)



Flip Flop D (FFD)

Costruito a partire da due Latch CD, presenta due ingressi C e D e due uscite Q e !Q. Il valore del segnale D viene messo in memorizzato quando C passa da 0 a 1. Quando C è costante o passa da 1 a 0, il FFD mantiene in memoria lo stato memorizzato al fronte di salita precedente.





CK=0 · Arriva ck=1 in M → M memorizza D

· Arriva ck = 0 in S → S non memorizza D

⇒ Q finale continua ad essere il valore precedente

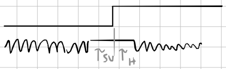
CK 0→1 · Arriva ck=0 in M → M non memorizza D, rimane lo stato precedente

· Arriva ck=1 in S → S memorizza lo stato precedente e lo dà in uscita

CK=1 · Continua ad essere la situazione precedente ⇒ S continua a fornire lo stesso valore di uscita. Stesso caso se ck torna a 0, si ripete il passaggio iniziale (Q continua ad essere l'ultimo valore memorizzato

CK (clock) è un segnale con periodo costante T.

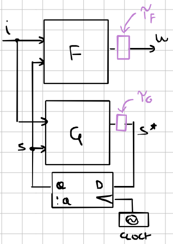
In merito al segnale D, affinché venga campionato fedelmente, deve essere stabile per un tempo $\tau_{SU} + \tau_H$



Tempo di setup τ_{SU} = tempo per cui il segnale D deve essere stabile prima del fronte

Tempo di Hold τ_H = tempo per cui il segnale D deve essere stabile dopo il fronte

RETI SEQUENZIALI SINCRONE



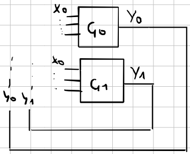
Se il periodo di clock T è > τ_G (tempo di elaborazione della rete G) allora, usando un FFD in retroazione, siamo in grado di "scandire il tempo", assicurandoci che lo stato attuale s non sia sfalsato.

Infatti, presupponiamo il FFD abbia in memoria uno stato s. G elabora, in un tempo τ_G, la configurazione che definisce lo stato futuro (es. 000 → 101). Tale configurazione (s^*) entra nel FFD (nella realtà ogni bit della configurazione entra in un FFD, come vedremo più avanti) (entra nel M del FFD). Al fronte di salita del clock, tale stato passa nel s del FFD, che lo rifornisce come stato attuale s. s entra in G, viene rielaborato in τ_G, al prossimo fronte di salita viene rilasciato il nuovo S*.

Grazie a tale meccanismo, eliminiamo corse critiche e alee statiche.

Vediamo come.

Problema della corsa critica:

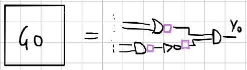


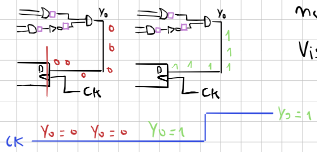
Supponiamo che $y_0, y_1 = 00$. All'ingresso $x_0 = 1$, y_0, y_1 devono diventare 11. G_0 è più veloce, per un certo istante dunque $Y_0 Y_1 = 01 \Rightarrow y_0 y_1 = 01$ rientra nel circuito di elaborazione. Se per l'ingresso $x_0 = 1$ lo stato 01 è stabile, allora si blocca in 01.

Con le configurazioni adiacenti facevamo si che un solo bit alla volta potesse cambiare, e tramite a.a.s primitivo facevamo si che ogni stato fosse stabile per una sola configurazione. Se però introduciamo un FFD per bit, con periodo di clock (sincronizzato) $T > T_a$, allora facciamo si che Y_0 e Y_1 si siano già trasformati entrambi prima di memorizzarli e dunque prima di farli passare $\Rightarrow$ non abbiamo più bisogno che gli stati abbiano configurazioni adiacenti.

Problema della alea statica:

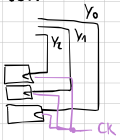
Come visto, tale problema riguarda i singoli bit di stato (le alee le eliminiamo nelle tdv delle singole variabili). Per elaborare Y_0 (l'uscita di G_0) parti diverse della rete di elaborazione potrebbero avere tempi diversi di elaborazione → Y_0 potrebbe momentaneamente glitchare a un valore indesiderato. Visto che però $T > T_a$, il FFD non memorizzerà (non lascerà passare) il glitch. A transitorio estinto, come sappiamo, le alee si estinguono, dunque a transitorio estinto Y_0 sarà uguale al valore desiderato (es. $Y_0 = 1$) $\Rightarrow$ nelle reti sequenziali sincrone non c'è bisogno





di termini anti-alea

Nota: per far sì che ciò avvenga tutti i FFD in retroazione devono essere sincronizzati



Dunque la sintesi di una rete sequenziali sincrona è identica in termini di passaggi a quella di una rete sequenziali sincrona ma:

- senza necessità di un dds primitivo, di configurazioni adiacenti e di alee statiche
- riportando un FFD su ogni bit in retroazione, tutti sincronizzati

RETE CON COMPORTAMENTO DI TIPO 2

Si ricorda che una rete di questo tipo, all'arrivo di un ingresso, produce un numero limitato di uscite.

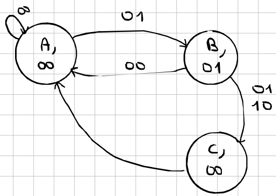
Si vuole ad esempio progettare una rete di Moore che, presi due ingressi x_0 e x_1, generi una sequenza di numeri a 2 bit. Nello specifico

ingresso 01 → uscita: sequenza 01 - 00 - 00

ingresso 10 → uscita: sequenza 01 - 01 - 00 - 00

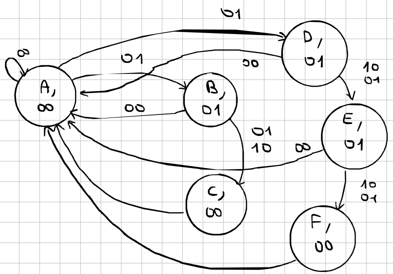
L'ingresso 11 non è consentito. Durante la generazione, ogni ingresso viene ignorato, eccetto 00 che annulla la sequenza. Al termine della sequenza, la rete genera 00 fino alla ricezione di ingressi 01 e 10

- dds (prima sequenza) (11 non può arrivare)



← a prescindere da cosa arrivi (eccetto 0 che annulla la sequenza) al ciclo di clock successivo B va in C, e lo stesso fa C.

- dds (prima più seconda sequenza)



- ecco la tabella

	00	01	11	10
A	A	B	–	D
B	A	C	–	C
C	A	A	–	A
D	A	E	–	E
E	A	F	–	F
F	A	A	–	A

z_1z_0
00
01
00
01
01
00

noto che due stati sono uguali $\longrightarrow$

	00	01	11	10
A	A	B	–	D
B	A	C	–	C
C	A	A	–	A
D	A	E	–	E
E	A	F	–	F
F	A	A	–	A

z_1z_0
00
01
00
01
01
00

$\downarrow$ posso dunque rimuoverne 1 e reindirizzare F a C

	00	01	11	10
A	A	B	–	D
B	A	C	–	C
C	A	A	–	A
D	A	E	–	E
E	A	C	–	C
~~F~~	~~A~~	~~A~~	~~–~~	~~A~~

z_1z_0
00
01
00
01
01
~~00~~

ora però altri due sono uguali $\longleftarrow$

	00	01	11	10
A	A	B	–	D
B	A	C	–	C
C	A	A	–	A
D	A	E	–	E
E	A	C	–	C
~~F~~	~~A~~	~~A~~	~~–~~	~~A~~

z_1z_0
00
01
00
01
01
~~00~~

$\downarrow$ ri-semplifico

	00	01	11	10
A	A	B	–	D
B	A	C	–	C
C	A	A	–	A
D	A	B	–	B
~~E~~	~~A~~	~~C~~	~~–~~	~~C~~
~~F~~	~~A~~	~~A~~	~~–~~	~~A~~

z_1z_0
00
01
00
01
~~01~~
~~00~~

Ho semplificato la tabella, e di conseguenza il d.d.s (che potrei ri-disegnare, vedi slide 63 foglio 3.4)

Esiste un modo algoritmico per la semplificazione. Questo processo è detto minimizzazione degli stati

- Costruisco la tabella delle implicazioni

	00	01	11	10	z_1z_0
A	A	B	–	D	00
B	A	C	–	C	01
C	A	A	–	A	00
D	A	E	–	E	01
E	A	F	–	F	01
F	A	A	–	A	00

B					
C					
D					
E					
F					
	A	B	C	D	E

ogni coppia è univoca (c'è AB, ma non c'è BA, ...)

	00	01	11	10	z_1z_0
A	A	B	–	D	00
B	A	C	–	C	01
C	A	A	–	A	00
D	A	E	–	E	01
E	A	F	–	F	01
F	A	A	–	A	00

B					
C					
D					
E					
F					
	A	B	C	D	E

B e A hanno uscite diverse non possono essere compatibili ⇒ metto una X

B	X				
C					
D					
E					
F					
	A	B	C	D	E

Teniamo a mente A e B incompatibili

	00	01	11	10	z_1z_0
A	A	B	–	D	00
B	A	C	–	C	01
C	A	A	–	A	00
D	A	E	–	E	01
E	A	F	–	F	01
F	A	A	–	A	00

B	X				
C					
D					
E					
F					
	A	B	C	D	E

A e C hanno la stessa uscita, dunque potrebbero essere compatibili. Affinché lo siano, gli stati futuri a parità d'ingresso devono essere compatibili.

Per l'ingresso 01, A → B, C → A ⇒ stati futuri A e B incompatibili (c'è la X messa prima nella tabella) ⇒ A e C incompatibili ⇒ metto una x

Posso ripetere il passaggio per ogni coppia di stati e completare la tabella.

Ad esempio, per BD noto che potrebbero essere compatibili se lo fossero C ed E ⇒

B	X				
C	X	X			
D	X	CE			
E	X				
F	X				
	A	B	C	D	E

⇒ vado avanti, in CE noto che C ed E non sarebbero compatibili ⇒ non lo sono neanche B e D ⇒ tolgo CE e metto una X

⇒

B	X				
C	X	X			
D	X	X	X		
E	X	CF	X		
F	X				
	A	B	C	D	E

⇒ noto che BE compatibili se lo sono CF ⇒ CF lo sono ⇒ metto dei trattini

⇒

B	X				
C	X	X			
D	X	X	X		
E	X	--	X	X	
F	X	X	--	X	X
	A	B	C	D	E

tutti gli altri, si può verificare, non lo sono

ritroviamo che C e F e B e D ←

E si possono "semplificare", come visto prima

⇒ si sceglie come semplificarli (si "identificano delle classi di compatibilità")

[A] → A

[B-E] → B

[C-F] → C

[D] → D

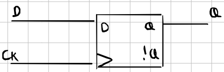
⇒ da qui si può poi ricostruire il d.d.s.

⇒

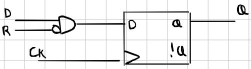
	00	01	11	10	$z_1 z_0$
A	A	B	–	D	00
B	A	C	–	C	01
C	A	A	–	A	00
D	A	B	–	B	01
~~E~~	~~A~~	~~C~~	~~–~~	~~C~~	~~01~~
~~F~~	~~A~~	~~A~~	~~–~~	~~A~~	~~00~~

FFD CON FUNZIONALITÀ AGGIUNTE

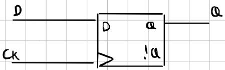
RESET: se 1, consente di riportare FFD a un valore iniziale (solitamente 0)



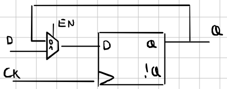
⇒



ENABLE: abilita il campionamento del FFD solo quando vale 1 (altrimenti il FFD mantiene in memoria il valore precedente)



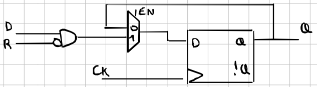
⇒



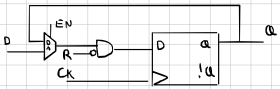
Grazie al MUX, se EN=1 allora viene selezionato l'ingresso D, altrimenti entra il vecchio ingresso (preso dalla retroazione)

Aggiungendo entrambe le funzionalità, possiamo creare due tipi di FFD:

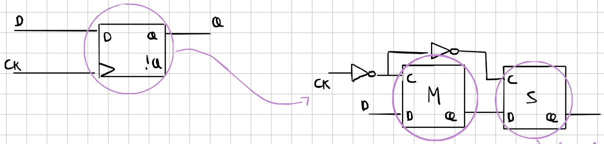
- **ENABLE prioritario**: anche se R=1, se ENABLE = 0 il FFD mantiene il valore precedente



- **RESET prioritario**: anche se ENABLE = 1, se RESET = 1 D viene ignorato e si salva 0 nel FFD



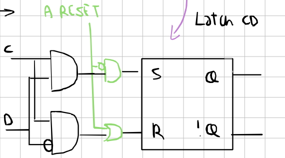
Il RESET visto agisce sul fronte di salita, ovvero il segnale viene resettato alla prima salita del clock dopo il RESET. In alcuni casi è necessario applicare il RESET immediatamente, senza aspettare la salita del clock. Dunque serve un **RESET asincrono**. Per realizzarlo occorre modificare la struttura interna del FFD



(anche M è fatto così

Latch CD

Reset asincrono, in questo modo se il RESET=1 forzo S a 0 e R a 1 ⇒ resetto il segnale (ignorando il clock in quanto agisco direttamente sul Latch)



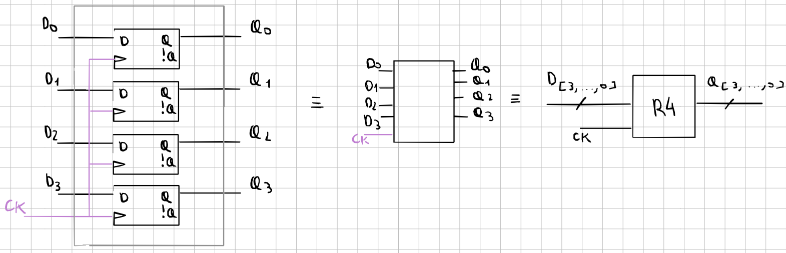
Latch SR

REGISTRO EDGE-TRIGGERED

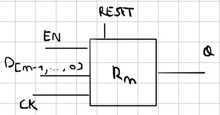
Dal momento che nei calcolatori l'unità minima di informazione considerata è il byte (8 bit) è necessario studiare componenti che operino con n bit in parallelo (n bit che "arrivano insieme").

Il registro edge-Triggered è una rete che consente la memorizzazione di n bit in parallelo.

Viene intuitivamente realizzato con una "batteria di FFD" sincronizzati, con un FFD per ogni bit da memorizzare



È anche possibile progettare registri con ENABLE, RESET e RESET ASINCRONO (vedi slide 14 foglio 3.5)



È possibile dimostrare tramite sintesi e analisi (slides 20, ..., 30) quanto visto prima, ovvero che aggiungere un segnale di ENABLE a un registro edge-Triggered si traduce nell'aggiungere un MUX2 all'ingresso dei FFD del registro, e aggiungere un segnale di RESET si traduce nell'aggiungere un AND all'ingresso dei FFD del registro.

Questo è vero e applicabile in qualsiasi rete, anche più complessa

REGISTRI A SCORRIMENTO (SHIFT REGISTER)

Supponiamo di voler trasmettere 256 bit.

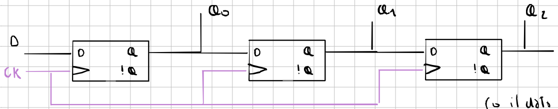
① Se li trasmetto in parallelo devo essere in grado di gestire 256 bit insieme
⇒ tanto spazio richiesto ma veloce

② Se li trasmetto uno alla volta (256 cicli di clock) devo allora gestire un solo bit alla volta (poco spazio richiesto) ma ci metto molto più tempo

Un compromesso spazio-tempo potrebbe essere trasmettere 8 bit in parallelo in 8 cicli di clock. Lo shift register riproduce il comportamento 2.

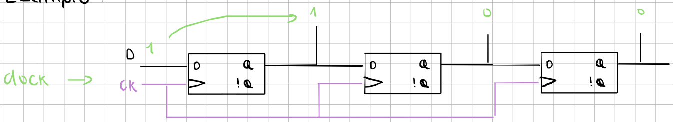
Un registro a scorrimento si presenta come una catena di FFD in cascata.

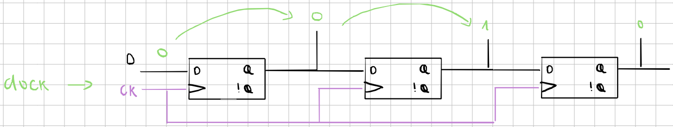
A ogni ciclo di clock, il bit in ingresso "scorre" verso destra

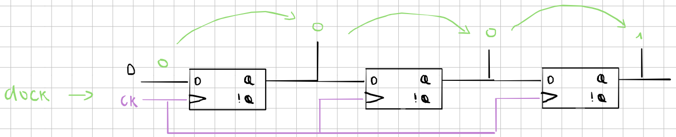


Esempio :

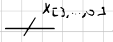
(o il dato che c'era memorizzato prima)



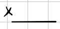




A ogni ciclo di clock il segnale scorre verso destra. Se ad esempio arriva un segnale a 4 bit (es. 1011) in 4 cicli di clock le uscite restituiranno tale segnale, che può essere letto da un processore che lavora con dati a 4 bit. Questo appunto ottimizza lo spazio (abbiamo visto un solo filo) ma richiede tempo (4 cicli di clock). Uno registro a scorrimento è dunque in grado a tutti gli effetti di memorizzare dati seriali (esempio di gemini del cavo ethernet e del processore)

Simbologia : 

= dati in parallelo

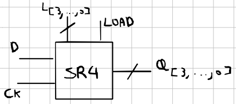


= un dato alla volta

Uno shift-register può essere costruito in diversi modi (SISO, PIPO, PISO, SIPO). (Parallel In - Serial Out, ...)

Inoltre, si può anche

- Aggiungere un segnale di LOAD, che abilita il caricamento di n bit alla volta, rispettivamente negli n FFD ⇒ salvataggio in parallelo (carica gli n bit che vengono passati ($L_{[n,\dots,0]}$) dentro agli n FLIP-FLOP)



- Aggiungere un segnale L/R (Left/Right). Se tale segnale è 0, lo spostamento avviene verso sinistra, altrimenti verso destra. △

Dunque i registri sono reti che permettono la memorizzazione di più di un bit alla volta. Ad esempio, un processore potrebbe voler trattare d bit alla volta ⇒ si può usare un registro edge-triggered. Se i bit vengono ad esempio uno dietro l'altro (es. da un cavo ethernet) per essere usati dalla CPU dovremmo arrivare d alla volta ⇒ in d cicli di clock, uno shift-register può memorizzare d bit e poi trasmetterli insieme volendo.

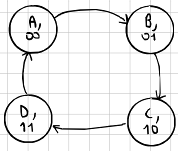
RETI SINCRONE E COMPORTAMENTI DI TIPO 3

Ricordiamo che questo comportamento descrive una rete che, all'arrivo di un ingresso, produce un numero illimitato di uscite.

Un esempio sono i contatori binari, reti di Moore che:

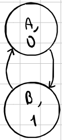
- hanno un diagramma degli stati ad anello, in cui ogni stato codifica un valore del conteggio ES: 00, 01, 10, 11, 00, 01, ...
- nella loro forme base non hanno ingressi ma iterano all'infinito automaticamente (detta rete free-running)
- il numero di stati determina la capacità massima del contatore (es. contatore da 0 (00) a 3 (11) => 4 bit => 4 stati

ES. a 4 bit

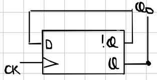


no ingressi, itera all'infinito

A DUE BIT (BC2)



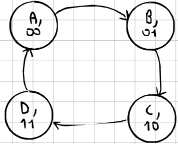
Schematico:



$Y_0 = !y_0$

← Ad ogni ciclo di clock la variabile di stato si inverte (0 → 1 → 0 →...)

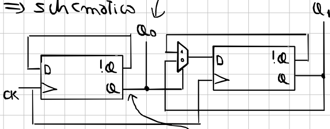
A 4 BIT (BC4)



y_0 si inverte in ogni passaggio di stato, y_1 si inverte ogni 2

=> schematico

(= Y_0) si inverte a ogni passaggio (come il precedente)



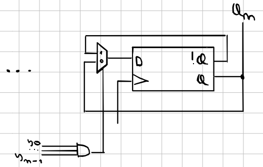
(≡ Y_1)

$Y_1 = y_1 !y_0 + !y_1 y_0$ ($y_1 \oplus y_0$)

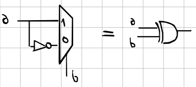
Q del primo FFD vale 1 ogni 2 cicli di clock => ogni due cicli fa passare !Q1, altrimenti Q1

BC 2^n

In generale, un BC2^n può essere realizzato come una catena di .

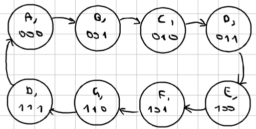


nota: vale inoltre che:



ES. sintesi di un BC8

TdT (e codifica stati)



→

	$y_2y_1y_0$		Z
A	000	001	000
B	001	010	001
C	010	011	010
D	011	100	011
E	100	101	100
F	101	110	101
G	110	111	110
H	111	000	111

con questa codifica le variabili di stato corrispondono alla rispettiva uscita ⇒ non c'è bisogno della mappa delle uscite

←

Mappa di Karnaugh

y_1y_0 \ y_2	0	1
00	1	1
01	0	0
11	0	0
10	1	1

Y_0

y_1y_0 \ y_2	0	1
00	0	0
01	1	1
11	0	0
10	1	1

Y_1

y_1y_0 \ y_2	0	1
00	0	1
01	0	1
11	1	0
10	0	1

Y_2

nota: le mappe di Karnaugh richiedono sempre l'adiacenza tra variabili

es:

	. . . . .
00	. . . . .
01	- - -
11	- - -
10	- -

↓ espressioni

$y_1y_0 \backslash y_2$	0	1
00	1	1
01	0	0
11	0	0
10	1	1

Y_0

$y_1y_0 \backslash y_2$	0	1
00	0	0
01	1	1
11	0	0
10	1	1

Y_1

$y_1y_0 \backslash y_2$	0	1
00	0	1
01	0	1
11	1	0
10	0	1

Y_2

MUX2 controllato da $y_1 y_0$

de Morgan $\Longrightarrow$ $y_2 \,!(y_1 y_0) + !y_2 (y_1 y_0)$

$Y_0 = !y_0$ $\quad$ $Y_1 = !y_1 y_0 + !y_0 y_1$ $\quad$ $Y_2 = !y_1 y_2 + y_1 y_0 !y_2 + !y_0 y_2$

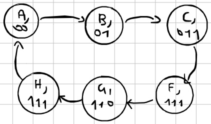
↳ MUX2 controllato da y_0

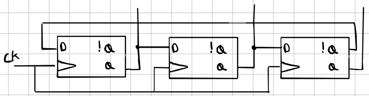
É anche possibile realizzare contatori asincroni (clock non sincronizzato).

Definizione: clock-gating = "alterare il segnale di clock"

Esistono anche:

- Contatori UP/DOWN, che permettono il conteggio crescente o decrescente (versi delle frecce nei D.D.S. invertiti) △
- Contatori di Johnson (JC), un contatore a riempimento/svuotamento che "aggiunge 1" in ogni FFD, e poi riporta tutti gli 1 (l'ultimo numero del contatore) a 0 (il primo numero del contatore). Si realizza con un anello chiuso di FFD. Non avendo gate, il ritardo è minimo => massima frequenza di clock. Tuttavia a parità di capacità desiderata servono più FFD per la realizzazione

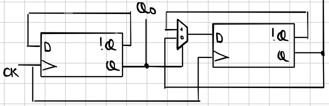




↳ con n FFD si può contare fino a 2n valori (non 2^n come visto precedentemente)

GENERATORI DI FORME D'ONDA

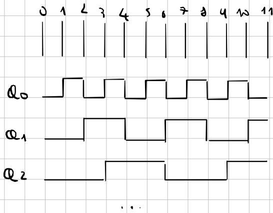
Q_1



← Contatore BC4

(commuta = cambia valore)

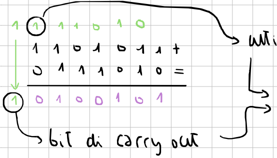
Abbiamo visto che Q0 cambia valore ogni ciclo di clock, Q1 ogni 2. Allo stesso modo, in un BC8 ($BC2^3$) avremmo un uscita Q_2 che commuta ogni 3 cicli di clock, e così via. Dunque un contatore è anche un generatore di forme d'onda:



← cicli di clock

RICHIAMI PRE-ESAME

OVERFLOW E COUT



ultimo riporto prima del carry out

1 se sono diversi

ultimo riporto ⊕ bit di carry-out = overflow

↳ es: 1 ⊕ 1 ⇒ 0 ⇒ no overflow

Per operazioni SENZA segno (non in complemento a 2 o analoghi) il bit che vale 1 se il risultato è errato è il carry-out

Per operazioni CON segno (in complemento a 2 o analoghi) il bit che vale 1 se il risultato è errato è l'overflow (ultimo riporto ⊕ bit di carry-out)

I termini anti-alea consentono di eliminare le alee statiche solo se gli ingressi cambiano uno alla volta

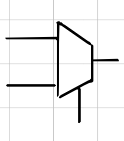
Δ = eventualmente da approfondire

Note:

- ~~se non compaiono, recuperare Half-adder e Full-adder~~
- eventualmente scrivere tutte le regole dell'algebra booleana
- approfondire se necessario (vedi le ex-verifiche) la questione overflow, carry out, carry in
- Eventualmente guardare meglio la ALU e la moltiplicazione / divisione per 2 (slides dopo la ALU
- Eventualmente riguardare cosa fa il d.d.s primitivo e cosa fa invece la risoluzione delle corse critiche
- Rivedere primo l. di De Morgan per la sintesi a P.S. (da segnare sul foglio)
- scrivere le forme d'onda nel caso di Moore e nel caso di Mealy
- scrivere sintesi a NAND e con MUX sul foglio
- capire se la sintesi a NAND scritta è giusta
- Ripassa bene anche le varie codifiche (BCD, complemento a due, ...)
- segnati le strutture interne dei blocchi (complete, con anche reset asincrono, enable, ...)
- se c'è spazio sul foglio, riportare il 7-segmenti
- approfondire come effettuare per bene la sintesi a MUX
- Ricorda che frasi del tipo "l'uscita cambia APPENA cambia l'ingresso" o simili indicano la necessità di una rete di Mealy
- Studiati bene i vari modi per formare un mux e la sintassi dell' espressione di un MUX
- Come determinare la massima frequenza
- Guarda l'esempio di Vims della verifica del 9/02/21 per una idea su come fare sintesi a MUX
- Nella sintesi a NAND trasforma anche le negazioni in NAND, ricordando che !x = x NAND x

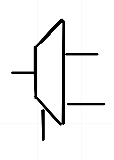
Blocchi

MUX



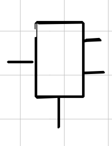
A seconda della configurazione degli N-Bit di controllo, seleziona un ingresso da far passare

DEMUX



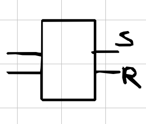
A seconda della configurazione degli N bit di controllo sceglie su quale uscita far arrivare l'ingresso

DECODER (DEC)



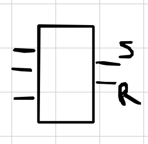
Ingresso fisso a 1 (o ENABLE), a seconda della configurazione degli N bit di controllo sceglie su quale uscita far arrivare l'ingresso (se I=1, è equivalente dire che sceglie quale uscita attivare)

HALF-ADDER (HA)



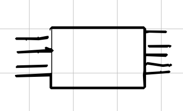
Prende n ingressi e restituisce la somma (S) e il resto (R) (realizzabile tramite sintesi della t.d.v.)

FULL-ADDER (FA)



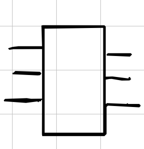
Prende N+1 ingressi (quelli da sommare + carry-in) e restituisce somma e resto (S e R)

ADDER A N BIT



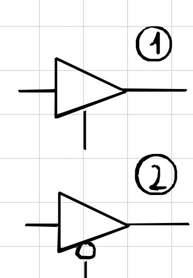
Prende due configurazioni a N bit e ne restituisce la somma e l'eventuale carry-out. È formato da N-1 FA e da 1 HA.

TRANSCODIFICATORE BINARIO-GRAY



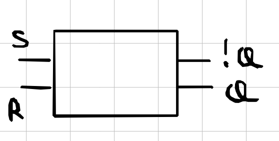
Prende un numero binario (una configurazione di bit in ingresso) e restituisce la configurazione Gray corrispondente (realizzabile tramite sintesi della t.d.v.)

BUFFER TRISTATE



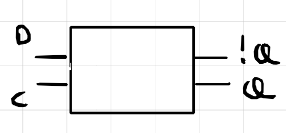
Prende un ingresso e lo lascia passare se il segnale di controllo è 1 (o 0 nel caso del buffer "attivo basso" (2))

LATCH SR (LSR)



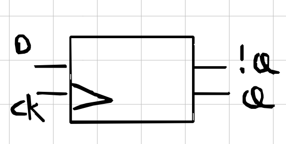
Prende in ingresso un segnale di set (S) e uno di reset (R). S e R non sono mai 11. Q e !Q sono l'uscita e l'uscita negata. Se S=1, Q viene impostata a 1, se R=1 l'uscita viene riportata a 0, se S=0 e R=0 l'uscita rimane quella precedentemente impostata

LATCH SD (LSD)



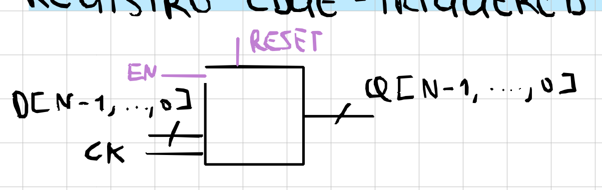
Si basa sul Latch SR. Se C=0, Q rimane come precedentemente impostata. Se C=1, Q diviene uguale a D.

FLIP-FLOP (FFD)



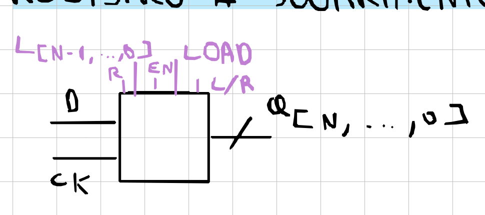
Costituito da due Latch SD, è in grado di campionare (dunque di "impostare Q") il valore di D solo quando CK passa da 0 a 1 (sul fronte di salita del clock)

REGISTRO EDGE-TRIGGERED (RN)



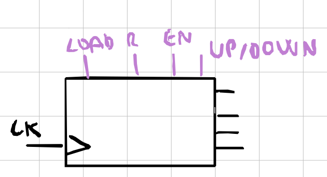
Consente la memorizzazione di più bit in parallelo. È formato da una pila di FFD in parallelo con clock sincronizzato. Il reset può anche essere asincrono, permettendo il reset senza aspettare il ciclo di clock successivo.

REGISTRO A SCORRIMENTO (SHIFT REGISTER) (SRN)



A ogni ciclo di clock, memorizza D e shifta (fa scorrere) il bit precedentemente memorizzato. In N cicli di clock riesce a memorizzare una sequenza di N bit. È composto da una cascata di FFD sincronizzati. Un eventuale segnale di LOAD, accompagnato da N "bit di load", dà la possibilità di caricare tali bit nei FFD. Un altro eventuale segnale è L/R, che permette lo scorrimento verso sinistra (Left) o destra (Right). Anche per lo shift register si può avere un segnale di EN e uno di RESET.

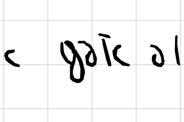
CONTATORE FREE RUNNING (BCN)



A ogni ciclo di clock, senza considerare gli ingressi, la rete passa nello stato successivo, che codifica un numero (contatore). Un contatore sincrono a 2^N bit (che può contare fino a 2^N) è costituito da a FFD sincronizzati opportunamente collegati. Un contatore asincrono (non considerabile come rete sincrona) è formato da una cascata di FFD opportunamente collegati in cui il clock dei FFD è dato dall'uscita del FFD precedente. Anche per questo dispositivo è possibile incorporare segnali di ENABLE, RESET e LOAD. È inoltre possibile integrare un meccanismo di UP/DOWN, che a seconda che sia UP (1) o DOWN (0) fa sì che la rete conti in ordine crescente o decrescente.

Essendo che ogni i-esimo bit di uscita cambia (commuta) ogni 2^i cicli di clock, ogni i-esimo bit di uscita crea un'onda di periodo $2 \cdot i$.

CONTATORE DI JOHNSON (JC)



Contatore in grado di contare $2 \cdot n$ valori riempiendo-svuotando i FFD che lo compongono. Ha il vantaggio di non avere gate al suo interno => frequenza di clock massima, ha lo svantaggio che richiede più FFD di un BC per essere costruito